# Finding the Memory Dependences Between GPU Kernels by Interpreting the Program's CUDA Source

Nafis Mustakin
University of California, Riverside

October 2026

**Abstract**

A CUDA program runs as a sequence of operations on the GPU: kernel launches, copies between host and device memory, and memory fills. One operation depends on an earlier one when it reads bytes the earlier one wrote, or writes bytes the earlier one read or wrote. A runtime system that knows these dependences can run independent operations at the same time, reorder them, or place their data, but in graph programs and other irregular code the bytes a kernel touches are chosen by values in memory, such as the neighbour lists of a graph, so a compiler cannot determine them from the source code. We present an interpreter that runs the program's CUDA source, host code and kernels together, on the CPU with the program's real input, and records which bytes every operation reads and writes. The interpreter runs all the threads of a kernel at once as arrays, one statement at a time, so a value loaded from memory, a branch on such a value, and an address computed from it are all known exactly. Two further mechanisms handle cases that a single run in this form would get wrong. When the threads of a kernel load pointers that name different buffers, the interpreter splits the threads into groups by buffer and runs each group separately. When the bytes a kernel touches depend on the order in which threads run, the interpreter runs the program under four legal thread orders and reports a dependence as certain only if every order produces it. We compare the interpreter against the bytes each operation actually touched on an NVIDIA A30 GPU, recorded with an NVBit tool we built for the purpose, on 13 test programs that include three breadth-first searches, a connected-components kernel and kernels that choose buffers through tables of pointers. Every dependence the interpreter reports as certain occurs on the GPU (1,650 certain dependences, none wrong), and the interpreter finds every dependence of 12 of the 13 programs. On connected components, where 60 runs on the GPU took 20 to 24 rounds and every traced run touched different bytes, it finds 0.981 to 0.995 of each run's dependences. Of the 1,427 dependences between kernels that pass through bytes chosen by data, 1,032 are reported as certain and all of those occur on the GPU. The cost is the interpreter's run time on the CPU, which for the inputs we measured is 0.3 to 624 times the program's whole run on the GPU. Most of that time goes to host loops that the interpreter runs one iteration at a time, and the kernels alone take 0.05 to 11.5 times the whole GPU run. Running all four thread orders costs 17.2 times as much as lockstep alone, on average. The exact bytes depend on the input, so they can be known only once the input is known, but the pattern of dependences between kernels was the same for three different graphs.

## 1 Introduction

A CUDA program is a host program, running on the CPU, that issues work to the GPU. The work arrives at the GPU as a sequence of *operations*: a *kernel launch* runs a function, the kernel, on many GPU threads at once, a *copy* moves bytes between host memory and device memory or within

device memory, and a *fill* (`cudaMemset`) sets a range of device memory to one value. By default the GPU runs these operations one after another, in the order the host issued them. That order is always safe, but it is often slower than necessary, because two operations that touch different bytes could run at the same time, and an operation whose inputs are ready could start before an unrelated operation finishes. CUDA lets a program express such freedom with streams and with task graphs [14], and research systems schedule GPU work from a dependence graph between kernels or parts of kernels [2, 1]. All of these need to know which operations depend on which.

An operation *depends* on an earlier one when they touch the same bytes and at least one of them writes them. For regular code, where every address is a simple function of the thread index and the sizes, a compiler can work out the bytes each kernel touches from the source code alone [8]. For irregular code it cannot. In a graph program the addresses a kernel touches are read from memory. For example, a breadth-first search visits `cost[edges[i]]`, where `edges` holds the graph, and it runs that statement only for vertices whose entry in a frontier array is set, an array that the previous kernel wrote. Which bytes the kernel touches is therefore a function of the input graph and of everything the program computed before the launch. Graph benchmark suites for GPUs consist mostly of such kernels [5, 4, 17].

In this work we determine these dependences by running the program's source code on the CPU, with its input, in an interpreter that records the bytes every operation touches. The interpreter extends MemPrint's interpreter for C and C++ programs [13], which runs a program's syntax tree from `main()` and keeps the values the program stores in memory, so that an address read from memory is known. We add the execution of kernels, the bookkeeping that turns touched bytes into dependences, and two mechanisms that irregular GPU code needs, and we measure the result against a GPU.

In this work, we make the following contributions:

- We describe an interpreter that runs a CUDA program's host code and kernels from source, runs all threads of a kernel together as arrays, and reports for every pair of operations whether they depend on each other, through which buffer, over how many bytes, and whether the dependence is certain or only possible (Section 3).
- We show how to handle a kernel whose threads use different buffers chosen through pointers stored in memory, by splitting the threads into groups by buffer (Section 3.6), and a kernel whose touched bytes depend on the order in which threads run, by running several legal thread orders and keeping as certain only what they agree on (Section 3.7).
- We build an NVBit [16] tool that records the bytes each operation touches on a real GPU, and a scoring method that compares two runs even when they issue a different number of operations (Section 4).
- We evaluate the interpreter on 13 programs on an NVIDIA A30: every certain dependence it reports occurs on the GPU, it finds all dependences in 12 of 13 programs, and it costs 0.3 to 624 times the program's whole run on the GPU, most of it in host code (Section 5).

## 2 Background and terms

This section defines the terms the rest of the paper uses. We assume only that the reader knows C.

**Threads, blocks and kernels.** A kernel is a C++ function marked `__global__`. The host launches it with a *grid* of $B$ *blocks*, each block a group of $T$ *threads*, and writes the launch as `k<<<B,T>>>(...)`. Every thread runs the whole function. A thread tells itself apart from the others through the built-in values `blockIdx` and `threadIdx`, and usually computes from them the element it works on. Threads of one block can share a small, fast memory (`__shared__` arrays) and wait for each other at a barrier (`__syncthreads()`). Threads of different blocks can communicate only through device memory, and the order in which blocks run is not specified. An *atomic* operation, such as `atomicAdd(&x, 1)`, reads and updates one memory location without another thread's update coming in between, and returns the old value.

**Operations and buffers.** We call every kernel launch, copy and fill an *operation*, and number the operations in the order the host issues them. A *buffer* is one region of device memory obtained from `cudaMalloc` or `cudaMallocManaged`. We measure memory in *units* of 4 bytes, so a dependence through one `int` or `float` is one unit.

**Dependences.** Operation $b$ depends on an earlier operation $a$ through a unit of a buffer if one of three things holds. In a *read after write* (RAW), $a$ is the last operation before $b$ that wrote the unit, and $b$ reads it, so $b$ needs the value $a$ produced. In a *write after write* (WAW), $b$ writes a unit that $a$ was the last to write, so $b$ must not finish before $a$, or the final value would be $a$'s. In a *write after read* (WAR), $b$ writes a unit that $a$ read after the last write, so $b$ must not overwrite it before $a$ has read it. We identify a dependence by its source operation, its destination operation, its kind and its buffer, and we record over how many units it passes.

**Certain and possible dependences.** A dependence is *certain* if the interpreter computed both accesses exactly, and, when it ran the program under several thread orders (Section 3.7), every order produced the dependence. It is *possible* if at least one of the two operations only perhaps touches it, for example because the access sits under a branch on a value the interpreter does not know, or at an address it could not compute. A tool that consumes the dependences can rely on a certain dependence, and must treat a possible one as present to stay safe.

**Needed dependences.** If $c$ depends on $b$ and $b$ depends on $a$, then running the operations in an order that respects the first two dependences also respects any dependence of $c$ on $a$. A scheduler therefore needs only the dependences that are not implied by a path through others. In graph terms these form the *transitive reduction* of the dependence graph, and we call them the *needed* dependences.

**Indirect accesses and data-dependent branches.** An access is *indirect* if its address is computed from a value loaded from memory, as in `a[b[i]]`. A branch is *data-dependent* if its condition reads memory, as in `if (mask[tid])`. A dependence is *data-dependent* if it passes through units that were touched by an indirect access or under a data-dependent branch. The interpreter marks these from the source code (Section 3.4). They are the dependences a compiler cannot determine, and the ones this paper is about.

Figure 1 shows both kinds in the kernel that expands the frontier in a breadth-first search, written in the style of Rodinia's `bfs` [6]. Which threads pass the test on `mask[tid]`, which elements of `edges` they read, and which elements of `cost` and `updating` they write all depend on the graph and on the previous levels of the search.

```
__global__ void expand(const Node *nodes, const int *edges, bool *mask, bool *updating,
                       const bool *visited, int *cost, int n) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    if (tid < n && mask[tid]) {                          // data-dependent branch
        mask[tid] = false;
        for (int i = nodes[tid].starting; i < nodes[tid].starting + nodes[tid].no_of_edges; i++) {
            int id = edges[i];                           // indirect: i comes from nodes[]
            if (!visited[id]) {                          // indirect, and a data-dependent branch
                cost[id] = cost[tid] + 1;                // indirect write
                updating[id] = true;
            }
        }
    }
}
```

**Figure 1:** The frontier expansion of a breadth-first search (`rodinia_bfs`, one of our test programs). The bytes the kernel reads and writes depend on the graph and on what earlier launches wrote.

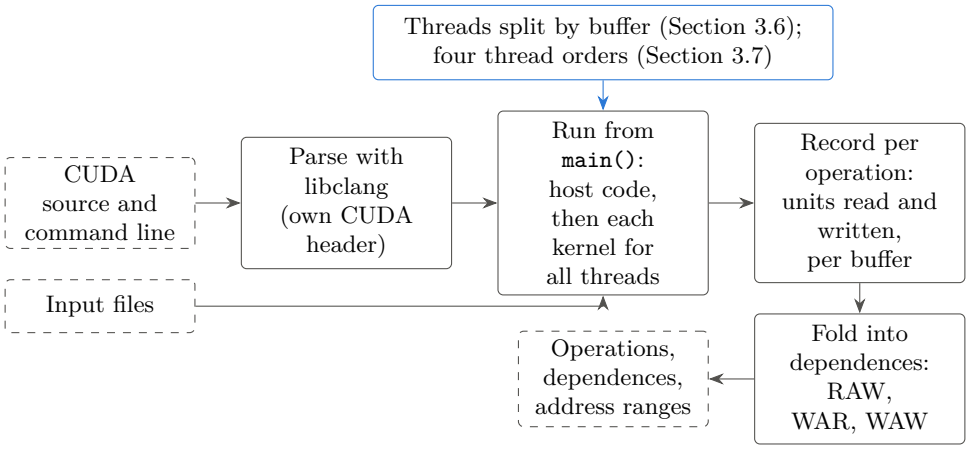



**Figure 2:** The steps of the interpreter. Nothing runs on the GPU.

# 3 The interpreter

## 3.1 Overview

The interpreter takes the program's CUDA source files and its command line, and runs the program on the CPU from `main()`. It does not compile the program and needs no GPU. Figure 2 shows the steps. First, libclang [12] parses the source into a syntax tree. CUDA's own headers are replaced by a small header of our own that declares the runtime functions (`cudaMalloc`, `cudaMemcpy` and so on), so no CUDA installation is needed. Second, the interpreter executes the syntax tree. Host code runs as in MemPrint's C++ interpreter [13], and each call to a CUDA runtime function or kernel launch becomes an operation. Third, each operation records the units it read and wrote in each buffer. Fourth, the operations are folded, in order, into the dependences between them. The output is a list of operations, a list of dependences with their kinds, buffers, unit counts and certainty, and optionally the address ranges each operation touched.

### 3.2 Running the host program

The host part of a CUDA program is ordinary C++, and the interpreter runs it the way MemPrint's C++ interpreter does [13]. Every allocation, global variable and the stack is a memory *object*. A pointer is a pair of an object and a byte offset into it. The interpreter keeps the integer value stored in every 4-byte unit of every object, and remembers which units hold pointers and to which object they point. As a result, when the program reads a graph from a file into an array and later indexes another array with an element of it, the interpreter knows the index. Floating-point values in memory are not kept, and a value the interpreter does not know is marked unknown rather than guessed.

Device memory fits into the same scheme. Each `cudaMalloc` creates a new object of kind *device*, and each `__device__` or `__constant__` variable is an object as well. A `cudaMemcpy` copies the stored values, and the pointer information, from one object to the other, so an index array built on the host and copied to the GPU keeps its contents, and a table of device pointers built on the host still points to the right buffers after the copy. A `cudaMemset` stores its value in the filled units.

A loop on the host that issues GPU work runs one iteration at a time, in program order. The interpreter otherwise runs short counted loops as one step over all iterations (Section 3.3), and we changed it so that an allocation, copy or fill inside such a loop is carried out once per iteration, because each iteration creates a different buffer or a different operation.

### 3.3 Running a kernel: all threads at once

A kernel launch runs the kernel's body once, for all of its threads together. Every variable of the kernel holds an array with one value per thread, and every statement is carried out for all threads before the next statement starts. MemPrint's interpreter uses the same representation to run all iterations of a loop at once, and for a kernel the threads take the place of the iterations. The thread's `threadIdx.x` is an array $0, 1, 2, \ldots$; an expression such as `blockIdx.x * blockDim.x + threadIdx.x` is computed for all threads with one array operation; and `edges[i]`, where `i` differs per thread, loads one element per thread from the stored values of the object `edges` points to.

A branch splits the threads. The interpreter evaluates the condition for every thread, runs the branch only for the threads where it holds, and merges the variables of the two groups afterwards. A loop whose trip count differs per thread runs until no thread continues, and a thread that returns from the kernel stops taking part. In this way every thread follows its own control path, decided by the values it loaded.

We call this order of execution *lockstep*: statement by statement, all threads at a time. Lockstep is one order in which the GPU could legally run the threads of a kernel that has no data races, because the GPU guarantees nothing about the relative progress of threads between barriers. A barrier, `__syncthreads()`, has no work to do in lockstep, since every thread has already reached it when the statement before it finishes. Each block has its own copy of the kernel's `__shared__` arrays. Atomic operations to the same address from different threads take effect one after another in thread order, and each thread receives the value that was there before its own update, as on the GPU. The interpreter runs the blocks of a large grid in groups of at most $2^{18}$ threads to bound its memory use.

### 3.4 Recording what each operation touches

Every operation records, per object and per byte, whether it read the byte and whether it wrote it. A kernel records every load and store that its threads carry out, a copy records the bytes it reads at

```
init<<<b, t>>>(a, 1.0f, n);
init<<<b, t>>>(b, 2.0f, n);
add<<<b, t>>>(a, b, c, n);     // c = a + b
scale<<<4, 64>>>(c, n);        // c *= 2
cudaMemcpy(h, c, n * 4,
           cudaMemcpyDeviceToHost);
```

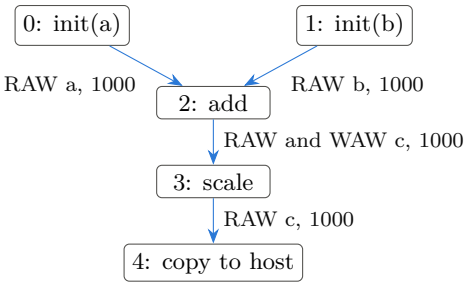



**Figure 3:** The operations of `chain` with $n = 1000$ and the dependences the interpreter reports, with the number of 4-byte units each passes through. The two `init` launches are independent and could run at the same time. All dependences are certain, and all are needed.

the source and writes at the destination, and a fill records the bytes it writes. The interpreter also records the host code between two operations as an operation of its own, which a tool can ignore; the evaluation leaves it out.

A record has two further marks. The first says whether the access is only possible, which is the case when the interpreter could not compute the access exactly. An address it could not compute covers the whole object it points into, and an access under a branch whose condition is unknown is recorded but marked as possible. An access through a pointer whose object is unknown is recorded as a possible access to every device buffer. The second mark says whether the access is data-dependent (Section 2). The interpreter decides this from the source code before the program runs, by marking every value that is loaded from memory, or computed from such a value, or assigned under a condition that is, and repeating until nothing changes. An access whose address or whose enclosing conditions use a marked value is data-dependent, and so is every access of a launch that the host issued from a loop whose condition depends on device data, such as a loop that runs until the GPU reports that nothing changed.

### 3.5 From touched bytes to dependences

The operations are folded into dependences in the order they were issued. For every unit of every object the interpreter keeps the last operation that certainly wrote it, the operations that possibly wrote it since then, and the operations that read it since then. When the next operation arrives, every unit it reads produces a RAW dependence on the last writer, every unit it writes produces a WAW dependence on the last writer and a WAR dependence on each reader, and then the operation's own writes replace the history of those units. A dependence is certain if both of its accesses are certain, and possible otherwise. The counts of units are summed per dependence. Figure 3 shows the result for a short program.

### 3.6 Threads that use different buffers

Some kernels choose a buffer, not only an element, from data. Figure 4 shows a kernel that distributes elements into one of four buckets, each a separate buffer, through a table of device pointers that the host copied to the GPU. In lockstep, the statement `bucket[k][...] = i` loads one pointer per thread, and the pointers of different threads point to different buffers. The interpreter's pointer is a pair of one object and an offset, so it cannot represent "buffer 0 for these threads and buffer 3 for those". Before the change described here, such a pointer became unknown, every write through it became a possible write to every device buffer, and every dependence from the kernel to the four consumers was only possible.

```
__global__ void distribute(const int *keys, int *const *bucket, int *count, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= n) return;
    int k = keys[i] % 4;
    bucket[k][atomicAdd(&count[k], 1)] = i;   // the buffer itself is chosen by keys[i]
}
```

**Figure 4:** A kernel that writes each element into one of four buffers, through a table of pointers in device memory (`ptrsel`). The four consumer kernels each read one buffer.

We resolve this by splitting the threads. When a kernel loads pointers from memory and the threads' pointers name different objects, the interpreter groups the threads by the object their pointer names, and then runs the statement, and the rest of the enclosing block of code, once for each group, with only that group's threads active. Inside a group the pointer names one object, so every access through it is exact. After the last group, the variables of all groups are merged as after a branch. A pointer variable that ends up naming different objects in different groups becomes unknown at the merge, which is safe.

Running a statement a second time is correct only if its first, interrupted execution changed nothing. The interpreter therefore counts stores and atomic operations, notes the count when each statement starts, and splits only if the count is unchanged when the pointer is loaded. If a store already happened, the pointer stays unknown as before. For `bucket[k][atomicAdd(...)]` the split happens, because C++17 evaluates the array expression `bucket[k]` before the index, so the pointer is loaded before the atomic operation [10]. A pointer loaded after an atomic operation in the same statement cannot be split. The interpreter also limits the split to at most 64 different objects.

### 3.7 Kernels whose result depends on thread order

Lockstep is a legal order of execution, but it is not the only one, and for some kernels the bytes touched depend on the order. The connected-components kernel in Figure 5 lowers labels in place. Thread $e$ reads the labels of the two ends of edge $e$ and writes the smaller into the larger with `atomicMin`. If another thread lowered one of those labels earlier in the same launch, thread $e$ reads the new value, otherwise the old one, so which threads write, and which units, depends on the order. The host repeats the kernel until a launch changes nothing, so the number of launches depends on the order too. On the GPU, 60 runs of this program took 20 to 24 rounds, and each of 10 traced runs touched different bytes. In lockstep, every thread reads its labels before any thread of the launch writes one, so no update reaches another thread within the same round, and the program takes 24 rounds. With lockstep alone, the interpreter reported all of its dependences as certain, and 8% to 24% of them did not occur in a given GPU run (Section 5.4).

To detect such kernels, the interpreter runs the whole program under several thread orders. All are legal on the GPU. The first is lockstep. In the other three, each block runs to completion before the next starts, in the order of block numbers, in reverse order, or in a random order. A later block then sees what earlier blocks wrote. We match the operations of the runs by their common beginning and their common end, because a loop that runs a different number of rounds adds or removes operations in the middle of the sequence. The operations before the loop and the first rounds match from the front, and the operations after the loop match from the back. A dependence stays certain only if every run has it between matched operations and every run reports it as certain. All other dependences of the first run, and the dependences that only the other runs

```
__global__ void hook(const int *src, const int *dst, int *label, int *changed, int m) {
    int e = blockIdx.x * blockDim.x + threadIdx.x;
    if (e >= m) return;
    int a = label[src[e]], b = label[dst[e]];   // may see another thread's update, or not
    if (a < b && atomicMin(&label[dst[e]], a) > a) *changed = 1;
    if (b < a && atomicMin(&label[src[e]], b) > b) *changed = 1;
}
```

**Figure 5:** One round of connected components by label propagation (`cc`). The host launches it until `changed` stays zero.

have between matched operations, become possible and are marked as depending on the schedule. A program whose kernels do not depend on the order, such as all three breadth-first searches in our tests, gives the same dependences under all four orders and loses nothing.

Four orders are a sample of the legal orders, not all of them, so a GPU run can still produce a dependence that none of the four produced. We return to this in Section 5.4.

### 3.8 What the interpreter cannot know

The interpreter needs the program's input, because the bytes an irregular kernel touches are a function of it. It does not model floating-point values stored in memory, the values of some library functions (for example `drand48`), or a pointer loaded in a statement after a store. Accesses that depend on such values are recorded as possible, so the dependences through them are possible too. The `maybe` test program checks this case. A kernel whose result depends on thread order is handled by sampling orders (Section 3.7), and a data race whose outcome no sampled order reproduces is not detected.

## 4 Measuring the truth on a GPU

To measure the interpreter's accuracy, we need the bytes each operation actually touches on a GPU. We built an NVBit [16] tool for this. NVBit is a framework that inserts code before chosen machine instructions of a kernel when the kernel is loaded. Our tool inserts, before every load, store and atomic operation on global memory, a call that sends the addresses of the 32 threads of the warp, the access size and whether it reads or writes, together with the number of the current launch, to a buffer that a CPU thread empties. On the host side the tool intercepts the CUDA driver's functions for allocation, copies, fills and kernel launches, so it knows every buffer's base address and size and every operation's position. It writes, per operation and buffer, the ranges of units read and written, in the same format the interpreter can write.

The tool was written against NVBit 1.7 on a machine without a GPU, and it needed three changes to work with NVBit 1.8 and CUDA 13.4. First, the GPU side of the buffer must be flushed explicitly after each launch and at the end, with a small kernel loaded as a separate module, otherwise the last partly filled buffer, which for a small program holds all records, is lost. Second, the launch number must be set before the instrumented code is enabled for the launch, otherwise the records carry a wrong number. Third, the CUDA 13 runtime may launch kernels through `cuLaunchKernelEx`, which the tool must intercept as well. In addition, the interpreter's stand-in CUDA header had to declare the launch function that clang uses when it finds a recent CUDA installation on the machine.

| Program | What it exercises | Lines | Kernels | Operations | Dependences |
|---|---|---|---|---|---|
| `chain` | elementwise kernels in a chain | 32 | 3 | 5 | 5 |
| `reduce` | a reduction through shared memory | 39 | 2 | 5 | 4 |
| `stencil` | a stencil repeated over two buffers | 22 | 1 | 7 | 16 |
| `scatter` | writes at indices built on the host | 35 | 3 | 6 | 5 |
| `fileidx` | indices and weights read from files | 41 | 2 | 5 | 6 |
| `histogram` | atomic updates of bins chosen by the data | 45 | 2 | 10 | 14 |
| `maybe` | a branch on values the interpreter does not model | 30 | 2 | 4 | 5 |
| `bfs` | breadth-first search, one kernel per level | 50 | 1 | 49 | 359 |
| `rodinia_bfs` | breadth-first search in the style of Rodinia | 103 | 2 | 59 | 609 |
| `wl_bfs` | breadth-first search with a frontier queue | 60 | 1 | 56 | 345 |
| `ptrsel` | a buffer chosen by the data through a pointer table | 35 | 2 | 10 | 21 |
| `ptrsel2` | pointer fields in structs, pointers in variables | 88 | 4 | 16 | 29 |
| `cc` | connected components, labels lowered in place | 49 | 2 | 76 | 685 |

**Table 1:** The test programs. Lines counts non-empty, non-comment lines. Operations is the number of kernel launches, copies and fills the program issues, and dependences the number of dependences on the GPU.

**Comparing two runs.** The traced run's bytes are folded into dependences by the same code as the interpreter's, so the two sides differ only in which bytes each operation touched. Buffers are matched by the order of their allocation, and operations by their order. When the two runs issue different numbers of operations, as `cc` does, operations are matched by common beginning and common end as in Section 3.7, and a traced dependence through an unmatched operation counts as missed. We report four scores. *Certain precision* is the share of the interpreter's certain dependences that the traced run has. *Recall* is the share of the traced run's dependences that the interpreter reports, certain or possible. *Certain recall* is the share it reports as certain. *Units error* is, over the dependences both have, the summed difference in unit counts divided by the traced units. We compute the same scores over the needed dependences of each side.

# 5 Evaluation

## 5.1 Setup

We ran the GPU programs on an NVIDIA A30 (compute capability 8.0) with driver 610.57, CUDA 13.4 and NVBit 1.8, and the interpreter on the same machine's CPU, with Python 3.10, NumPy and libclang 18.1. The programs were compiled with `nvcc -O2`. Table 1 lists the 13 test programs. They are small, between 22 and 103 lines of code, and each exercises one pattern: regular elementwise kernels, shared memory, scattered writes through host-built or file-provided indices, atomics, values the interpreter does not model, three versions of breadth-first search (one kernel per level, Rodinia's two-kernel form with a graph file, and a frontier queue filled with `atomicAdd`), buffers chosen through pointer tables, and connected components. The graphs have 4,096 vertices with 1 to 5 random neighbours each. We wrote the programs together with the interpreter, so they test the patterns it was built for; Section 7 discusses what this does not show.

## 5.2 Accuracy of the dependences

Table 2 compares the interpreter, run under all four thread orders, with one traced run of each program. *Every dependence the interpreter reports as certain occurs on the GPU*: the certain precision is 1.00 for all 13 programs, over 1,650 certain dependences. The interpreter finds every

| | Dependences | | | | | | | Address ranges | | |
|---|---|---|---|---|---|---|---|---|---|---|
| Program | Certain | Possible | Certain precision | Recall | Certain recall | Needed cert. precision | Needed recall | Exact sets | Unit precision | Unit recall |
| `chain` | 5 | 0 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 8/8 | 1.00 | 1.00 |
| `reduce` | 4 | 0 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 7/7 | 1.00 | 1.00 |
| `stencil` | 16 | 0 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 11/11 | 1.00 | 1.00 |
| `scatter` | 5 | 0 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 9/9 | 1.00 | 1.00 |
| `fileidx` | 6 | 0 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 8/8 | 1.00 | 1.00 |
| `histogram` | 14 | 0 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 17/17 | 1.00 | 1.00 |
| `maybe` | 3 | 2 | 1.00 | 1.00 | 0.60 | 1.00 | 1.00 | 7/8 | 0.94 | 1.00 |
| `bfs` | 359 | 0 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 108/108 | 1.00 | 1.00 |
| `rodinia_bfs` | 609 | 0 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 195/195 | 1.00 | 1.00 |
| `wl_bfs` | 345 | 0 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 172/172 | 1.00 | 1.00 |
| `ptrsel` | 21 | 0 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 | 25/25 | 1.00 | 1.00 |
| `ptrsel2` | 23 | 34 | 1.00 | 1.00 | 0.79 | 1.00 | 0.90 | 31/49 | 0.57 | 1.00 |
| `cc` | 240 | 466 | 1.00 | 0.98 | 0.35 | 1.00 | 1.00 | 151/170 | 0.99 | 0.99 |

**Table 2:** The interpreter against one traced run per program. Certain and Possible count the interpreter's dependences. Precision and recall are defined in Section 4; "Needed" columns score only the needed dependences of each side. The address-range columns compare, for every operation, buffer, and read or write, the set of units touched: Exact sets counts how many of these sets are identical, and unit precision and recall are the shares of the interpreter's units the GPU touched and of the GPU's units the interpreter reported.

traced dependence of 12 programs, and 0.98 of those of `cc`. In 10 of the 13 programs it reports every dependence as certain, and the unit counts match exactly.

The three exceptions are the programs built to contain what the interpreter cannot resolve exactly. In `maybe`, a branch on `drand48` values makes two dependences possible, and one of them did not occur on the GPU. In `ptrsel2`, one kernel loads its pointer after an atomic operation in the same statement, so its threads cannot be split (Section 3.6), and its accesses are possible accesses to every device buffer; the other three kernels of the program are exact. In `cc`, the dependences of rounds whose touched bytes depend on thread order are possible (Section 5.4).

### 5.3 Dependences through data-dependent addresses and branches

Table 3 restricts the comparison to dependences between two kernels that pass through bytes touched by an indirect access or under a data-dependent branch, the dependences a compiler cannot determine. There are 1,427 of them across the programs. The interpreter reports 1,032 as certain, and all of these occur on the GPU. In the three breadth-first searches, every one of the 868 such dependences is certain and correct, including those through the frontier queue of `wl_bfs`, whose slots threads claim with `atomicAdd` in an order that differs between runs. The order changes which vertex lands in which slot, but not the set of slots written or the set of vertices whose distances the next level reads, so the dependences between launches do not change. In `ptrsel`, the dependence of each consumer on the bucket it reads is certain, with the exact number of elements, and the consumer of the bucket that stays empty has no dependence on the producer. The remaining dependences of this kind are the possible ones of `maybe`, `ptrsel2` and `cc` described above.

**Address ranges.** The right part of Table 2 compares the units each operation touched, not only the dependences. For 11 programs every set of units is identical to the GPU's. The interpreter's sets are larger only where it reports possible accesses: `maybe` (512 units possibly written, 254 written), `ptrsel2` (the kernel whose pointer could not be split covers every buffer) and `cc`.

| Program | Between kernels | Through data-dependent bytes | Reported certain | Certain and in the trace | In the trace (any) |
|---|---|---|---|---|---|
| `scatter` | 1 | 1 | 1 | 1 | 1 |
| `fileidx` | 1 | 1 | 1 | 1 | 1 |
| `histogram` | 1 | 1 | 1 | 1 | 1 |
| `maybe` | 2 | 1 | 0 | 0 | 1 |
| `bfs` | 233 | 233 | 233 | 233 | 233 |
| `rodinia_bfs` | 454 | 454 | 454 | 454 | 454 |
| `wl_bfs` | 181 | 181 | 181 | 181 | 181 |
| `ptrsel` | 6 | 6 | 6 | 6 | 6 |
| `ptrsel2` | 22 | 11 | 2 | 2 | 6 |
| `cc` | 538 | 538 | 153 | 153 | 509 |
| Total | 1452 | 1427 | 1032 | 1032 | 1393 |

**Table 3:** Dependences between two kernels (copies and fills excluded), and among them those through bytes reached by an indirect access or under a data-dependent branch. The last two columns count how many of the latter the traced run has. Programs with none are omitted.

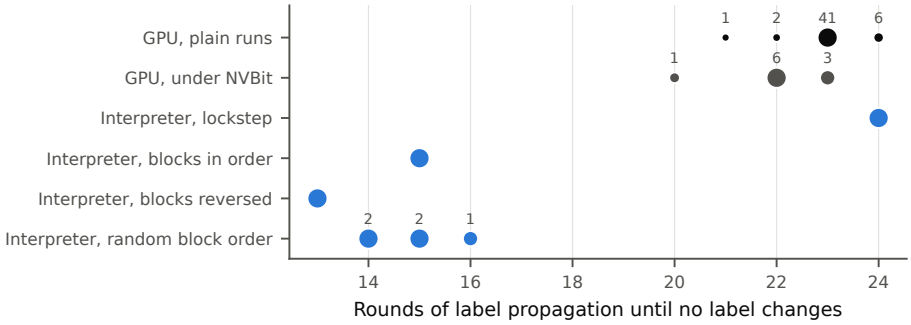



**Figure 6:** Rounds of `cc` until no label changes, on the GPU and in the interpreter. Each dot is a round count, its size and the number above it give how many runs produced it. The random order was run with five seeds.

**How much the exact ranges add.** In our programs, most kernels touch all or most of every buffer they use, so tracking whole buffers instead of exact units gives the same dependences for 10 of the 13 programs (Table 4). Exact ranges matter in `ptrsel` and `ptrsel2`, where several kernels touch disjoint parts of shared buffers, and they remove one false pair in `scatter` and `wl_bfs`. The test programs therefore show that the interpreter finds the exact bytes, but not that exact bytes are needed; programs whose kernels partition buffers, such as blocked matrix factorisations, would show that.

### 5.4 Kernels whose result depends on thread order

Figure 6 shows how many rounds `cc` takes. On the GPU, 50 plain runs and 10 runs under NVBit took between 20 and 24 rounds. The interpreter's orders took 13 to 24 rounds. Lockstep takes the most rounds, because no label update reaches another thread within a round. Running blocks one after another lets labels flow from earlier blocks to later ones within a round, which takes fewer rounds. The GPU, which runs many blocks at once but not all of them, lies between the two, inside the interpreter's range.

| | Exact units | | One interval | | Whole allocation | |
|---|---|---|---|---|---|---|
| Program | Pairs | Needed | Pairs | Needed | Pairs | Needed |
| `chain` | 5 | 4 | 5 | 4 | 5 | 4 |
| `reduce` | 4 | 4 | 4 | 4 | 4 | 4 |
| `stencil` | 17 | 6 | 17 | 6 | 17 | 6 |
| `scatter` | 6 | 5 | 7 | 5 | 7 | 5 |
| `fileidx` | 5 | 4 | 5 | 4 | 5 | 4 |
| `histogram` | 13 | 9 | 13 | 9 | 13 | 9 |
| `maybe` | 4 | 3 | 4 | 3 | 4 | 3 |
| `bfs` | 916 | 48 | 916 | 48 | 916 | 48 |
| `rodinia_bfs` | 1009 | 69 | 1009 | 69 | 1009 | 69 |
| `wl_bfs` | 1185 | 55 | 1190 | 55 | 1191 | 55 |
| `ptrsel` | 17 | 13 | 18 | 13 | 21 | 9 |
| `ptrsel2` | 26 | 18 | 26 | 18 | 27 | 15 |
| `cc` | 2328 | 75 | 2328 | 75 | 2328 | 75 |

**Table 4:** Pairs of operations that conflict in the traced run (both touch a unit and one writes it), and the needed pairs among them, when each operation's touched units are taken exactly, as one interval per buffer, or as the whole buffer.

| Program | Interpreter | Certain | Possible | Certain precision | Recall | Certain recall | Needed cert. recall | Units error |
|---|---|---|---|---|---|---|---|---|
| `ptrsel` | Full interpreter | 21 | 0 | 1.00 | 1.00 | 1.00 | 1.00 | 0.00 |
| `ptrsel` | Without thread splitting | 18 | 14 | 1.00 | 1.00 | 0.86 | 0.50 | 0.00 |
| `ptrsel2` | Full interpreter | 23 | 34 | 1.00 | 1.00 | 0.79 | 0.81 | 0.07 |
| `ptrsel2` | Without thread splitting | 16 | 93 | 1.00 | 1.00 | 0.55 | 0.10 | 0.47 |
| `cc` | Full interpreter | 240 | 466 | 1.00 | 0.98 | 0.35 | 0.53 | 0.12 |
| `cc` | Lockstep schedule only | 703 | 0 | 0.96 | 0.98 | 0.98 | 1.00 | 0.03 |

**Table 5:** The interpreter with one mechanism turned off, against the same traced runs as Table 2.

Figure 7 scores the interpreter against each of the 10 traced runs. With lockstep alone, the interpreter reports all of its dependences as certain, and 0.76 to 0.92 of them occur in a given run, so the rest are false. With the four orders merged, the certain precision is 1.00 against every run, and the interpreter reports 0.36 to 0.45 of each run's dependences as certain and 0.981 to 0.995 as certain or possible. The certain ones are mostly those of the early rounds and of the setup, which every order shares. A run's dependences that the interpreter misses come from orders that none of the four produced, for example a label written in one round and next overwritten three rounds later.

### 5.5 What each mechanism contributes

Table 5 turns each of the two mechanisms off in turn. Without splitting threads by buffer, every dependence through the buckets of `ptrsel` becomes possible, so only 18 of the 21 traced dependences are reported as certain and only half of the needed ones; in `ptrsel2`, certain recall falls from 0.79 to 0.55 and certain recall of the needed dependences from 0.81 to 0.10. Recall stays at 1.00, because the unknown pointer makes the accesses possible accesses to every buffer, which includes the right one. Without the extra thread orders, `cc` reports false certain dependences, as Section 5.4 showed.

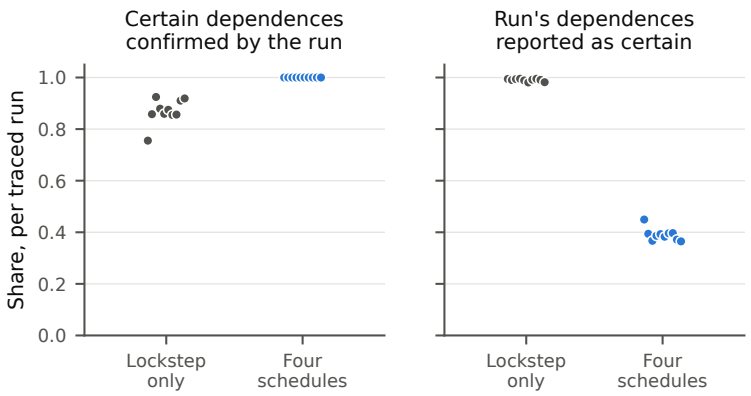



**Figure 7:** `cc` against each traced run: the share of the interpreter's certain dependences that the run has (left), and the share of the run's dependences that the interpreter reports as certain (right). Lockstep alone reports false certain dependences, and the four orders together do not.

## 5.6 Overhead

The interpreter's cost is its run time on the CPU. Figure 8 and Table 6 compare it with the program's own run on the GPU and with the run under the NVBit tool, for six programs at three input sizes each. The GPU runs take 0.31 to 0.37 seconds, and almost all of that time is the start-up of the CUDA runtime, because at these sizes the kernels themselves take milliseconds. Against the whole GPU run, the interpreter in lockstep takes 0.3 to 624 times as long, and the NVBit tool 2.0 to 13.8 times. Against the kernels alone, the interpreter is several orders of magnitude slower, as expected from simulating every thread in Python on one CPU core.

*Most of the interpreter's time is spent in host code, not in kernels.* The "In kernels" column of Table 6 gives the share of the time spent running kernels. The kernels of `chain`, with 3,000,448 GPU threads, take 4.0 seconds, and those of `rodinia_bfs`, with 2,228,224 threads, take less than one second, so `rodinia_bfs` as a whole takes 2.8 seconds, against 2.2 seconds under NVBit. The time spent in kernels alone is 0.05 to 11.5 times the whole GPU run, with a median of 0.6. The programs that take longest, `wl_bfs` (214 seconds, of which 0.24 in kernels) and `bfs` (122 seconds, of which 0.8 in kernels), spend up to 99.9% of their time in host loops that carry a value from one iteration to the next. These loops are the random-number generator that builds the graph in `wl_bfs` and `cc`, and a running maximum over the result in `bfs`. The interpreter runs all iterations of a loop at once only when the iterations do not depend on each other, so it runs these loops one iteration at a time, at about a millisecond per iteration. Such loops belong to the test harness here, not to the computation on the GPU, but real programs contain them as well, for example when they read or generate their input.

Running all four thread orders costs 3.9 to 94 times as much as lockstep alone (17.2 on average). Where host code dominates, the cost is close to four times, because each order runs the host code once more. Where kernels dominate, the orders that run one block at a time cost much more than lockstep, because they carry out every statement once per block instead of once per group of up to $2^{18}$ threads. A tool could run the extra orders only for kernels that read memory that other threads of the same launch write, which is where the order can change the result. We have not done so.

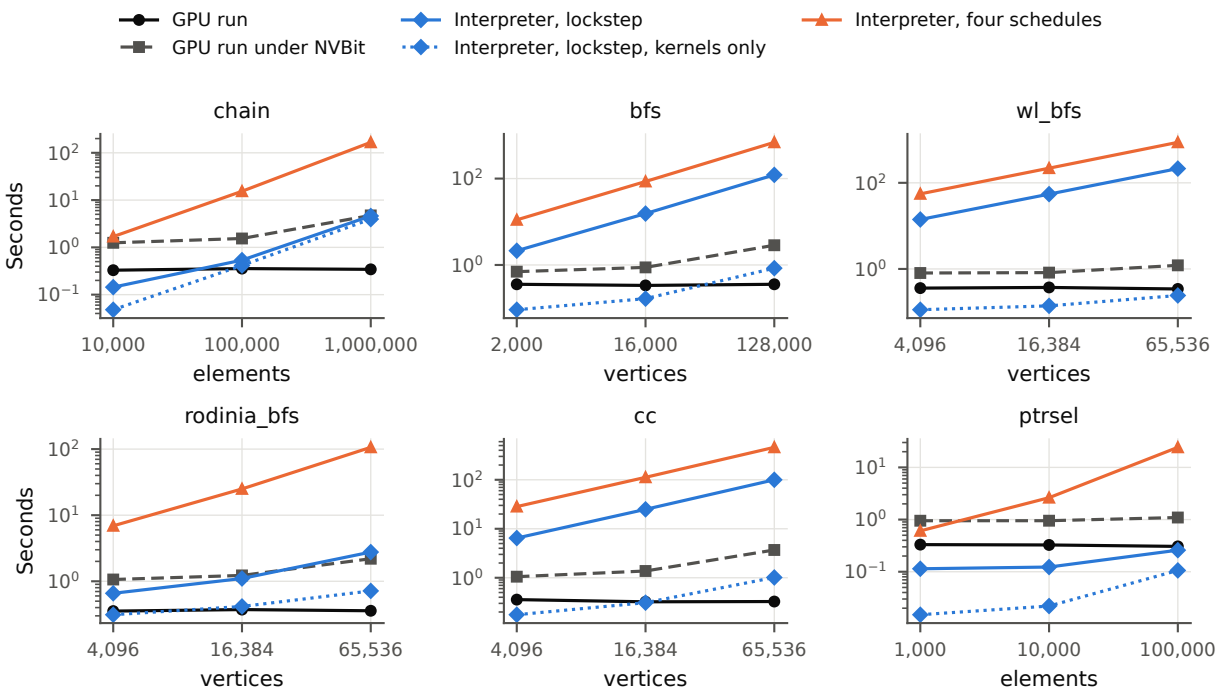



**Figure 8:** Wall-clock time against input size, both axes logarithmic: the program on the GPU (CUDA start-up included), the program under the NVBit tool, the interpreter in lockstep, the part of that time spent running kernels, and the interpreter under all four thread orders together.

### 5.7 Before the input is known

The interpreter needs the input. To see what depends on the input and what does not, we ran `rodinia_bfs` on three random graphs (Table 7). The searches took 51 to 63 operations and produced 417 to 656 dependences, and only 48 dependences have the same operations and units in all three graphs, most of them through the one-unit flag that the host copies back after every round. The exact bytes are therefore a property of the input. We then describe each dependence by a *pattern*: the names of its two operations, its kind, its buffer, and how many operations apart they are, for example "`expand` then `commit` one operation later, RAW through `updating`". All 24 patterns of the needed dependences are the same for the three graphs. The structure of the dependences is therefore a property of the program, and only the number of rounds and the units are a property of the input. A tool that must decide before the input is known could use the patterns, and refine them with the interpreter once the input is available. We have not built such a tool. Run without the input, the interpreter today stops after one round of the search, because the loop's exit condition comes from the GPU and is unknown, and it reports 15 of the 22 patterns that connect operations at most six apart, nearly all of them as possible.

## 6 Related work

**Static analysis of GPU kernels.** Compilers determine the dependences of loops and kernels whose addresses are affine functions of loop indices with the polyhedral model [8, 9], but an address read from memory falls outside that model. GPUVerify [3] proves that a kernel is free of data races for all inputs, and GKLEE [11] explores a kernel's behaviour with symbolic inputs. Both reason

| | | | | GPU (s) | | Interpreter | | | |
|---|---|---|---|---|---|---|---|---|---|
| Program | Input | GPU threads | Operations | Run | Under NVBit | Lockstep (s) | In kernels | Four schedules (s) | Lockstep / GPU run |
| `chain` | 10,000 | 30,592 | 5 | 0.33 | 1.25 | 0.14 | 33% | 1.7 | 0.4 |
| `chain` | 100,000 | 300,544 | 5 | 0.36 | 1.54 | 0.54 | 77% | 15.4 | 1.5 |
| `chain` | 1,000,000 | 3,000,448 | 5 | 0.34 | 4.74 | 4.62 | 86% | 166.5 | 13.5 |
| `bfs` | 2,000 | 30,720 | 49 | 0.36 | 0.70 | 2.14 | 4% | 11.1 | 6.0 |
| `bfs` | 16,000 | 288,000 | 58 | 0.34 | 0.88 | 15.48 | 1% | 85.6 | 46.1 |
| `bfs` | 128,000 | 2,816,000 | 70 | 0.36 | 2.86 | 121.90 | 1% | 690.3 | 341.4 |
| `rodinia_bfs` | 4,096 | 106,496 | 59 | 0.35 | 1.06 | 0.66 | 47% | 6.9 | 1.9 |
| `rodinia_bfs` | 16,384 | 524,288 | 71 | 0.37 | 1.23 | 1.10 | 38% | 25.0 | 3.0 |
| `rodinia_bfs` | 65,536 | 2,228,224 | 75 | 0.36 | 2.19 | 2.77 | 26% | 106.3 | 7.7 |
| `wl_bfs` | 4,096 | 5,120 | 56 | 0.36 | 0.81 | 14.08 | 1% | 55.4 | 39.2 |
| `wl_bfs` | 16,384 | 16,128 | 59 | 0.37 | 0.82 | 54.64 | 0% | 218.7 | 146.9 |
| `wl_bfs` | 65,536 | 60,288 | 71 | 0.34 | 1.21 | 214.14 | 0% | 871.0 | 624.3 |
| `cc` | 4,096 | 102,400 | 76 | 0.36 | 1.05 | 6.46 | 3% | 28.3 | 18.1 |
| `cc` | 16,384 | 409,600 | 76 | 0.32 | 1.37 | 24.92 | 1% | 112.2 | 77.2 |
| `cc` | 65,536 | 1,835,008 | 85 | 0.33 | 3.69 | 99.95 | 1% | 459.5 | 305.7 |
| `ptrsel` | 1,000 | 5,120 | 10 | 0.33 | 0.95 | 0.11 | 13% | 0.6 | 0.3 |
| `ptrsel` | 10,000 | 50,560 | 10 | 0.33 | 0.95 | 0.12 | 18% | 2.6 | 0.4 |
| `ptrsel` | 100,000 | 500,480 | 10 | 0.31 | 1.09 | 0.26 | 41% | 24.2 | 0.8 |

**Table 6:** Run times. GPU threads is the total number of threads of all launches. The GPU columns are wall-clock times of the whole program, CUDA start-up included (median of three runs). "In kernels" is the share of the lockstep interpreter's time spent running kernels; the rest runs host code. The last column divides the lockstep interpreter's time by the GPU run.

about one kernel and treat memory contents as arbitrary, so neither tells which bytes a kernel touches on a given input, and neither relates one launch to the next. Our interpreter answers the latter question, for one input.

**Dynamic analysis.** NVBit [16] and race detectors such as BARRACUDA [7] observe the accesses of a running program on the GPU, as our ground-truth tool does. They give the exact bytes of one run, but only by running the program on the GPU with instrumentation, and only after the fact. The interpreter gives the same information before the program runs on the GPU, from the source and the input.

**Inspector and executor.** Runtime parallelisation of irregular loops runs an *inspector* that reads the index arrays to compute the loop's dependences, and then an *executor* that runs the loop in an order that respects them [15]. The interpreter is an inspector for a whole CUDA program. For accesses whose indices come only from the input, an inspector can be cheap, but in the breadth-first searches the indices depend on values earlier kernels computed, so finding the dependences means computing those values, which the interpreter does.

**Dependence-driven GPU scheduling.** Wireframe [2] and BlockMaestro [1] run GPU work from a dependence graph between thread blocks of consecutive kernels, and CUDA graphs [14] let the programmer state dependences between operations. They need the dependences as input. BlockMaestro derives them from the kernels' memory accesses when these are regular; the

| Input | Vertices | Edges | Operations | Dependences | Needed | Same bytes in all graphs | Needed patterns in all graphs |
|---|---|---|---|---|---|---|---|
| graph A | 4,096 | 12,184 | 59 | 610 | 155 | 48 | 24/24 |
| graph B | 4,096 | 12,289 | 63 | 656 | 168 | 48 | 24/24 |
| graph C | 8,192 | 41,222 | 51 | 417 | 129 | 48 | 24/24 |
| no input | – | – | 11 | 56 | 37 | – | 15/22 |

**Table 7:** `rodinia_bfs` on three graphs. "Same bytes in all graphs" counts dependences with the same operations, kind, buffer and unit count in all three. A pattern is a dependence without its position: the names of its operations, kind, buffer and distance in operations.

interpreter could supply them for irregular kernels when the input is known.

**Irregular GPU benchmarks.** Rodinia [6], Pannotia [5], the LonestarGPU programs [4] and Gunrock [17] contain the patterns of our test programs: frontier arrays, frontier queues filled with atomics, and label propagation. We have not yet run the interpreter on these suites.

# 7 Limitations

A limitation of the evaluation is that the test programs are small and were written together with the interpreter, which means that they show that the mechanisms work on the patterns they target, not how often real programs contain patterns the interpreter does not support. Unmodified benchmark suites will use library calls, templates, streams and larger inputs that the tests do not.

A limitation of the interpreter is that it needs the program's input, which means that its exact results are for that input only (Section 5.7).

A limitation of the thread orders is that they are four samples of the legal orders, which means that a GPU run of a kernel whose result depends on the order can produce dependences the interpreter misses (0.5% to 1.9% of the traced dependences of `cc`).

A limitation of splitting threads is that it applies only to a pointer loaded before any store in its statement, which means that a pointer loaded after an atomic operation in the same statement still makes every access through it a possible access to every buffer.

A limitation of the cost is that the interpreter simulates every GPU thread on one CPU core in Python, which means that its run time grows with the number of threads and with sequential host loops, and that the three extra thread orders cost more than lockstep because they run one block at a time (Section 5.6).

# 8 Conclusion

We showed that the memory dependences between the operations of a CUDA program, including those that pass through bytes chosen by data, can be found before the program runs on the GPU, by interpreting its source on the CPU with its input. Running all threads of a kernel at once as arrays keeps the cost proportional to the work of the kernels, splitting threads by the buffer their pointers name makes accesses through pointer tables exact, and running several legal thread orders keeps the reported certain dependences correct when a kernel's result depends on the order. Against an NVIDIA A30, every certain dependence the interpreter reported occurred, and it missed dependences only in a kernel whose behaviour changed from run to run. The next steps

are to run the interpreter on unmodified graph benchmarks at full input sizes, and to derive the input-independent pattern of dependences without the input.

## References


[1] AmirAli Abdolrashidi, Hoda Aghaei Esfeden, Ali Jahanshahi, Kaustubh Singh, Nael Abu-Ghazaleh, and Daniel Wong. BlockMaestro: Enabling programmer-transparent task-based execution in GPU systems. In *Proceedings of the 48th Annual International Symposium on Computer Architecture (ISCA)*, pages 333–346, 2021.

[2] AmirAli Abdolrashidi, Devashree Tripathy, Mehmet Esat Belviranli, Laxmi Narayan Bhuyan, and Daniel Wong. Wireframe: Supporting data-dependent parallelism through dependency graph execution in GPUs. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 600–611, 2017.

[3] Adam Betts, Nathan Chong, Alastair F. Donaldson, Shaz Qadeer, and Paul Thomson. GPUVerify: A verifier for GPU kernels. In *Proceedings of the ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, pages 113–132, 2012.

[4] Martin Burtscher, Rupesh Nasre, and Keshav Pingali. A quantitative study of irregular programs on GPUs. In *Proceedings of the IEEE International Symposium on Workload Characterization (IISWC)*, pages 141–151, 2012.

[5] Shuai Che, Bradford M. Beckmann, Steven K. Reinhardt, and Kevin Skadron. Pannotia: Understanding irregular GPGPU graph applications. In *Proceedings of the IEEE International Symposium on Workload Characterization (IISWC)*, pages 185–195, 2013.

[6] Shuai Che, Michael Boyer, Jiayuan Meng, David Tarjan, Jeremy W. Sheaffer, Sang-Ha Lee, and Kevin Skadron. Rodinia: A benchmark suite for heterogeneous computing. In *Proceedings of the IEEE International Symposium on Workload Characterization (IISWC)*, pages 44–54, 2009.

[7] Ariel Eizenberg, Yuanfeng Peng, Toma Pigli, William Mansky, and Joseph Devietti. BARRACUDA: Binary-level analysis of runtime RAces in CUDA programs. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 126–140, 2017.

[8] Paul Feautrier. Dataflow analysis of array and scalar references. *International Journal of Parallel Programming*, 20(1):23–53, 1991.

[9] Tobias Grosser, Armin Größlinger, and Christian Lengauer. Polly — performing polyhedral optimizations on a low-level intermediate representation. *Parallel Processing Letters*, 22(4), 2012.

[10] International Organization for Standardization. *Programming Languages — C++, ISO/IEC 14882:2017*, 2017. Section 8.2.1 [expr.sub]: the expression $E1[E2]$ is sequenced with $E1$ before $E2$.

[11] Guodong Li, Peng Li, Geof Sawaya, Ganesh Gopalakrishnan, Indradeep Ghosh, and Sreeranga P. Rajan. GKLEE: Concolic verification and test generation for GPUs. In *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, pages 215–224, 2012.

[12] The LLVM Project. *libclang: C Interface to Clang.* `https://clang.llvm.org/doxygen/group__CINDEX.html`.

[13] Nafis Mustakin. From source code to sampling correction: Predicting memory-footprint models for unseen programs. Technical report, University of California, Riverside, 2026. MemPrint report, `paper_generalization/` in the MemPrint repository.

[14] NVIDIA Corporation. *CUDA C++ Programming Guide, Release 13*, 2026. Sections on streams and CUDA graphs.

[15] Joel H. Saltz, Ravi Mirchandaney, and Kay Crowley. Run-time parallelization and scheduling of loops. *IEEE Transactions on Computers*, 40(5):603–612, 1991.

[16] Oreste Villa, Mark Stephenson, David Nellans, and Stephen W. Keckler. NVBit: A dynamic binary instrumentation framework for NVIDIA GPUs. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 372–383, 2019.

[17] Yangzihao Wang, Andrew Davidson, Yuechao Pan, Yuduo Wu, Andy Riffel, and John D. Owens. Gunrock: A high-performance graph processing library on the GPU. In *Proceedings of the 21st ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*, 2016.